



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klfh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 2

Project Title: Linux Process Creation, Execution and Termination System

Team Members

Roll Number	Student Name	Signature
2520090001	Kamtam Priyanka	<i>K. Priyanka</i>
2520090123	Jalla Geya Reddy	<i>Geya</i>

1. Abstract

Processes are one of the most important concepts in an operating system because they represent programs that are currently being executed. Understanding how a process is created, executed, monitored, and terminated helps in understanding how the operating system manages system resources and provides services to applications.

This project focuses on developing a Linux-based system that demonstrates the complete lifecycle of a process. The system will use C programming and Linux/POSIX system calls such as `fork()`, `exec()`, `waitpid()`, and `exit()` to demonstrate process creation, execution, synchronization, and termination. The project will also display important process information such as Process ID (PID), Parent Process ID (PPID), process status, and the relationship between parent and child processes.

The project aims to provide a practical understanding of the process abstraction and the interaction between user programs and the operating system through system calls. It will also demonstrate how a parent process creates and controls child processes and how the operating system maintains process states during execution.

The expected outcome is a simple Linux-based process management system through which users can create processes, execute programs, observe parent-child relationships, and monitor process termination.

2. Problem Statement

When a program is executed in Linux, several operating system mechanisms work behind the scenes to create and manage the corresponding process. For beginners, concepts such as `fork()`, `exec()`, process IDs, parent-child relationships, process states, and `waitpid()` can be difficult to understand only through theory.

The problem addressed by this project is the lack of a simple practical system through which students can observe the complete process lifecycle in Linux.

The proposed project provides a controlled environment for creating child processes, executing different programs, monitoring their execution, and handling their termination. It demonstrates how user-space programs interact with kernel services using system calls.

This project is relevant to Operating Systems and Systems Programming because it directly applies concepts from CO-1: OS as a Service Layer and CO-2: Processes and Process Control, particularly process abstraction, system calls, process creation, execution, synchronization, and termination.

3. Objectives

1. To understand and demonstrate process creation in Linux using the `fork()` system call and observe the relationship between parent and child processes.
2. To demonstrate program execution using the `exec()` family of system calls and understand how a newly created process can replace its program image with another program.
3. To monitor and synchronize processes using `wait()/waitpid()` and observe different stages of the process lifecycle.
4. To demonstrate process termination and status handling using mechanisms such as `exit()` and process exit status, while displaying useful information such as PID and PPID.

4. Proposed Methodology

The project will be implemented as a Linux-based C program that provides a simple interface for demonstrating process management operations.

The basic flow of the system will be:

User Input → Parent Process → `fork()` → Child Process → `exec()` → Program Execution → Parent waits using `waitpid()` → Child Terminates → Parent Displays Status

Initially, the application will run as a parent process and provide options to perform different process operations. When the user selects process creation, the program will call `fork()` to create a child process.

The parent and child processes will then follow their respective execution paths. The child process can use an `exec()` system call to execute another Linux program. Meanwhile, the parent process can use `waitpid()` to wait for the child and synchronize its execution.

The application will display information such as the PID of the parent, PID of the child, PPID, execution status, and termination status. This will help users clearly observe the parent-child relationship and process lifecycle.

Error handling will also be included for system calls such as `fork()`, `exec()`, and `waitpid()` so that failures are handled properly instead of causing unexpected behavior.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Abstraction	Represents a running program and helps understand how Linux manages processes.
<code>fork()</code>	Creates a new child process from the parent process.
<code>exec()</code> family	Replaces the child process's program with another executable program.
<code>waitpid()</code>	Allows the parent process to wait for and synchronize with a child process.
<code>exit()</code>	Terminates a process and provides an exit status to the parent.
<code>getpid()</code>	Retrieves the Process ID of the current process.
<code>getppid()</code>	Retrieves the Parent Process ID.
Process Lifecycle	Demonstrates creation, execution, waiting, and termination states.
Parent-Child Relationship	Shows how processes are related after <code>fork()</code> .
System Calls	Demonstrates how a user-space C program requests services from the Linux kernel.
Process Synchronization	Ensures that the parent appropriately waits for the child process.
Process Exit Status	Allows the parent to determine how the child process terminated.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520090001	Kamtam Priyanka	Design the overall process management flow, implement process creation using <code>fork()</code> , process information display using <code>getpid()/getppid()</code> .
2520090123	Jalla Geya Reddy	Implement process execution and termination using <code>exec()</code> and <code>waitpid()</code> , handle process status, errors, testing, and demonstration.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Primary operating system environment for implementing and testing Linux process-management features.
C	Main programming language used for systems programming and Linux system calls.
GCC	Compiler used to compile the C programs.
Linux System Calls	Provide access to operating system services such as fork(), exec(), and waitpid().
Bash / Linux Terminal	Used to compile, execute, and test the project.
Git	Used for version control and tracking project development.
GitHub	Used to store and manage the project source code and documentation.

8. Expected Outcome

The project is expected to produce a working Linux-based application that demonstrates the basic process lifecycle in a practical and easy-to-understand manner.

The application will allow the user to observe how a parent process creates a child process using fork(), how the child executes another program using exec(), and how the parent synchronizes with the child using waitpid().

The system will display useful process information such as PID, PPID, execution status, and termination status. It will also demonstrate how system calls provide a connection between a user-space program and Linux kernel services.

The final project will help students understand process creation, execution, synchronization, and termination through actual Linux execution rather than only theoretical examples.

9. GitHub Repository

GitHub Repository Link:

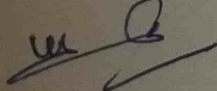
<https://github.com/geyaareddyj/KLH-CSIT-2029-2-Linux-Process-Management/tree/main>

Repository Created: ☒ Yes ☐ No

10. Faculty Name and Signature

Faculty Name: M. Raghupathi

Signature:



Remarks:

11. Faculty Use

Project Approved: ☒ Yes ☐ No

Date: 2/9/2026

Faculty Signature: 